Travaux pratiques

Machine Learning pour la Logistique

Responsable Module : **Oussama Boussaid**
Niveau : **Bachelor 2 ILOG**
Année : **2026 / 2027**

TP Manipulation de données logistiques avec NumPy « LogiStock »

INFORMATIONS GÉNÉRALES

Module	Machine Learning pour la Logistique
Durée	3 h 30
Nature du TP	**Travaux pratiques sur machine** : chaque notion du cours est rappelée puis implémentée directement dans un notebook Jupyter
Thématique	NumPy : création de tableaux, dimensions, indexation, découpage, redimensionnement, recherche, filtrage
Environnement	Google Colab ou Jupyter (Python 3, NumPy, matplotlib)
Travail en	Individuel
Livrables (étudiants)	**Notebook complété** (`TP_NumPy_LogiStock_Etudiant.ipynb`) avec les réponses aux questions

CONTEXTE PROFESSIONNEL

Vous venez d'intégrer l'équipe **données** de l'entrepôt **« LogiStock »**, un prestataire logistique qui stocke douze familles de produits dans quatre zones (A, B, C, D) et réalise chaque mois plusieurs centaines d'expéditions. Le responsable d'exploitation souhaite s'appuyer sur ses fichiers de données pour repérer les ruptures de stock, suivre l'activité mensuelle et comprendre l'origine des retards de livraison.

Avant d'entraîner des modèles de Machine Learning, il faut savoir **manipuler les données sous forme de tableaux**. C'est le rôle de la bibliothèque NumPy, sur laquelle reposent Pandas et Scikit-Learn, utilisés dans la suite du module.

Votre mission : reprendre, notion par notion, le cours d'introduction à NumPy et l'appliquer immédiatement aux données de LogiStock. Vous écrivez uniquement du code NumPy ; les graphiques sont fournis dans le notebook pour vous permettre de **visualiser** le résultat de chaque manipulation.

Données fournies

Les trois fichiers CSV sont générés par la première cellule du notebook (ils sont également fournis séparément), puis chargés dans des tableaux NumPy.

Fichier	Tableau(x) NumPy	Contenu
stocks_entrepot.csv	produits (1-D), stock (2-D)	Quantité en stock de 12 produits dans 4 zones (A, B, C, D)
livraisons.csv	livraisons (2-D)	200 livraisons : identifiant, distance (km), poids (kg), délai (h), coût (EUR)
expeditions_mensuelles.csv	expeditions (1-D)	Nombre d'expéditions par mois sur 24 mois (2 années)

OBJECTIFS PÉDAGOGIQUES

À la fin de ce TP, vous serez capable de :

- Expliquer l'intérêt de NumPy par rapport aux listes Python
- Créer des tableaux `ndarray` de dimension 0 à 3 et déterminer leur nombre de dimensions
- Accéder à un élément par **indexation** et extraire une partie d'un tableau par **découpage**
- Modifier la structure d'un tableau avec `reshape()`
- Rechercher des éléments avec `np.where()` et **filtrer** un tableau avec des booléens
- Combiner ces outils pour analyser des données logistiques réelles (stocks, expéditions, livraisons)

Organisation de la séance

Chaque partie suit le même déroulé : **rappel de cours**, **exemple guidé** (cellule complète à exécuter), **travail demandé** (cellules à compléter), **visualisation** fournie, puis **questions**.

Partie	Durée
PARTIE 0 : Prise en main : pourquoi NumPy ?	15 min
PARTIE 1 : Création de tableaux et dimensions	25 min
PARTIE 2 : Indexation des tableaux	25 min
PARTIE 3 : Découpage de tableaux (slicing)	35 min
PARTIE 4 : Redimensionnement de tableaux (reshape)	25 min
PARTIE 5 : Recherche dans les tableaux avec where()	30 min
PARTIE 6 : Filtrage par tableau de booléens	30 min
PARTIE 7 : Synthèse : audit de l'entrepôt LogiStock	25 min

PARTIE 0 : Prise en main : pourquoi NumPy ? (15 min)

0.1 Rappel de cours

- NumPy (« **Numerical Python** ») est une bibliothèque Python utilisée pour manipuler des tableaux. Elle propose aussi des fonctions d'algèbre linéaire, de transformée de Fourier et de calcul matriciel.
- NumPy a été créé en 2005 par Travis Oliphant ; c'est un projet open source, utilisable librement.
- Contrairement aux listes, les tableaux NumPy sont stockés dans un **espace mémoire contigu** : l'accès et les calculs sont très efficaces.
- Les parties de NumPy qui demandent des calculs rapides sont écrites en C ou en C++, et optimisées pour les processeurs récents.
- NumPy est généralement importé sous l'alias `np` : `import numpy as np`.

0.2 Exemple guidé

```
# Calcul du carré de 1 000 000 de nombres : liste Python puis tableau NumPy
nombres_liste = list(range(1000000))
nombres_numpy = np.arange(1000000)      # np.arange : équivalent NumPy de range

debut = time.time()
carres_liste = []
for n in nombres_liste:
    carres_liste.append(n * n)
temps_liste = time.time() - debut

debut = time.time()
carres_numpy = nombres_numpy * nombres_numpy    # un seul calcul sur tout le tableau
temps_numpy = time.time() - debut

print("Temps avec une liste  :", round(temps_liste, 4), "s")
print("Temps avec NumPy      :", round(temps_numpy, 4), "s")
print("NumPy est environ", round(temps_liste / temps_numpy), "fois plus rapide")
```

```
Sortie :
Temps avec une liste  : 0.1335 s
Temps avec NumPy      : 0.0057 s
NumPy est environ 23 fois plus rapide
```

Les temps mesurés varient d'une machine à l'autre ; seul l'ordre de grandeur compte.

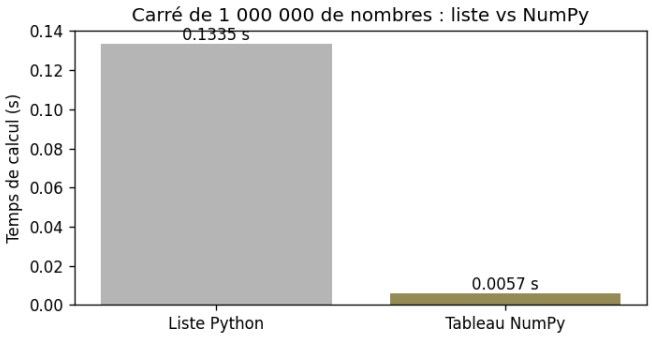


Figure : Temps de calcul : liste Python et tableau NumPy

Travail demandé

1. Exécutez les trois cellules de préparation du notebook (génération des CSV, outils de visualisation, chargement des données) et observez les tableaux affichés.

2. Affichez la version de NumPy installée, disponible dans `np.__version__`.
3. Mesurez le temps de calcul de la **somme** du million de nombres : `sum(nombres_liste)` pour la liste, `nombres_numpy.sum()` pour le tableau. Stockez les deux durées dans `temps_somme_liste` et `temps_somme_numpy`.

Questions (à rédiger dans le notebook)

Q0.1 : Pourquoi un tableau NumPy est-il plus rapide qu'une liste Python pour ce type de calcul ?

Q0.2 : Que permet l'écriture `import numpy as np` ?

PARTIE 1 : Création de tableaux et dimensions (25 min)

1.1 Rappel de cours

- L'objet tableau de NumPy s'appelle `ndarray`. On le crée avec la fonction `np.array()`, à partir d'une liste.
- **Dimension 0** (scalaire) : une seule valeur. Chaque valeur d'un tableau est un tableau de dimension 0.
- **Dimension 1** : un tableau dont les éléments sont des scalaires (un vecteur).
- **Dimension 2** : un tableau dont les éléments sont des tableaux de dimension 1 (une matrice : lignes et colonnes).
- **Dimension 3** : un tableau dont les éléments sont des matrices.
- L'attribut `ndim` renvoie un entier : le nombre de dimensions du tableau.

1.2 Exemple guidé

```
arr = np.array([1, 2, 3, 4, 5])
print(arr)
print(type(arr))

a = np.array(42)                                    # 0-D
b = np.array([1, 2, 3, 4, 5])                       # 1-D
c = np.array([[1, 2, 3], [4, 5, 6]])                # 2-D
d = np.array([[[1, 2, 3], [4, 5, 6]], [[1, 2, 3], [4, 5, 6]]])   # 3-D

print(a.ndim)
print(b.ndim)
print(c.ndim)
print(d.ndim)
```

```
Sortie :
[1 2 3 4 5]
<class 'numpy.ndarray'>
0
1
2
3
```

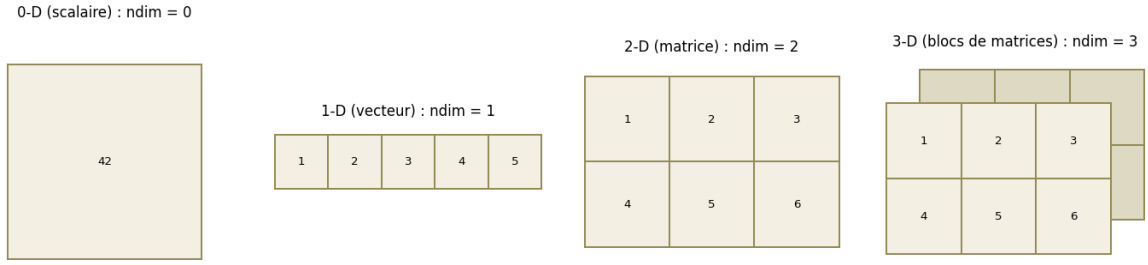


Figure : Représentation des tableaux a, b, c et d de l'exemple

Travail demandé

4. Créez le tableau 1-D `capacites` contenant la capacité (en palettes) des zones A, B, C et D : 500, 350, 420, 280. Affichez le tableau, son type et son nombre de dimensions.
5. Créez le tableau 2-D `stock_mini` : 2 produits (lignes) et 3 zones (colonnes), avec les valeurs 40, 15, 0 pour le premier produit et 22, 8, 31 pour le second. Affichez son nombre de dimensions.
6. Créez le tableau 3-D `planning` (nombre de camions chargés) : 2 semaines, 2 équipes par semaine, 3 jours par équipe. Semaine 1 : équipe 1 = 12, 15, 11 ; équipe 2 = 9, 14, 10. Semaine 2 : équipe 1 = 13, 16, 12 ; équipe 2 = 8, 11, 9. Affichez son nombre de dimensions.
7. Affichez le nombre de dimensions des tableaux chargés depuis les CSV : `expeditions`, `stock` et `livraisons`.

Questions (à rédiger dans le notebook)

Q1.1 : Quel est le type renvoyé par `type(capacites)` ? En quoi diffère-t-il d'une liste ?

Q1.2 : Pour le tableau `stock`, que représente chacune des deux dimensions ?

PARTIE 2 : Indexation des tableaux (25 min)

2.1 Rappel de cours

- Indexer un tableau, c'est accéder à un élément en indiquant son numéro d'index. **Les index commencent à 0.**
- Tableau 1-D : `arr[i]`.
- Tableau 2-D : `arr[ligne, colonne]`.
- Tableau 3-D : `arr[bloc, ligne, colonne]`.
- Un index négatif compte depuis la fin : `arr[-1]` est le dernier élément.

2.2 Exemple guidé

```
arr = np.array([1, 2, 3, 4])
print(arr[0])
print(arr[-1])

arr = np.array([[1, 2, 3, 4, 5], [6, 7, 8, 9, 10]])
print("2e élément de la 1re ligne :", arr[0, 1])

arr = np.array([[[1, 2, 3], [4, 5, 6]], [[7, 8, 9], [10, 11, 12]]])
print(arr[0, 1, 2])
```

```
Sortie :
1
4
2e élément de la 1re ligne : 2
6
```

Travail demandé

8. Dans `expeditions`, récupérez le volume du premier mois (`premier_mois`), du sixième mois (`sixieme_mois`) et du dernier mois avec un index négatif (`dernier_mois`).
9. Dans `stock`, récupérez dans `quantite_cible` la quantité du produit d'index 4 en zone C (colonne d'index 2). Affichez aussi le nom du produit avec `produits[4]`.
10. Calculez dans `total_palettes_AB` la quantité totale du produit d'index 0 dans les zones A et B (addition de deux éléments indexés).
11. Dans `planning`, récupérez dans `camions` le nombre de camions de la semaine 2, équipe 1, jour 3.

Visualisation attendue

Une fois les tâches terminées, exécutez la cellule de visualisation fournie dans le notebook. Vous devez obtenir :

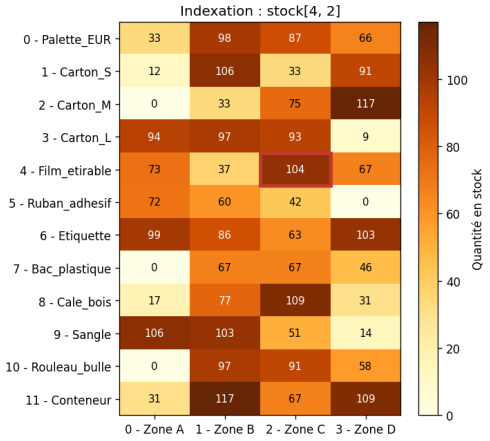


Figure : Carte du stock : la case encadrée correspond à stock[4, 2]

Questions (à rédiger dans le notebook)

Q2.1 : Pourquoi l'instruction `stock[12, 0]` provoque-t-elle une erreur ?

Q2.2 : Écrivez, avec des index négatifs, l'accès au stock du dernier produit dans la dernière zone.

PARTIE 3 : Découpage de tableaux (slicing) (35 min)

3.1 Rappel de cours

- Le découpage fonctionne comme pour les listes : on extrait les éléments compris entre deux index avec la notation `[début:fin:pas]`.
- L'index `fin` est **exclu**. Valeurs par défaut : `début` = 0, `fin` = longueur du tableau, `pas` = 1.
- Tableau 2-D : on donne une plage pour les lignes et une pour les colonnes, `arr[lignes, colonnes]`.
- Le symbole `:` seul signifie « tout » : `arr[:, 0]` renvoie toute la première colonne.

3.2 Exemple guidé

```
arr = np.array([1, 2, 3, 4, 5, 6, 7])
print(arr[1:5])
print(arr[4:])
print(arr[::2])

arr = np.array([[1, 2, 3, 4, 5], [6, 7, 8, 9, 10]])
print(arr[0:2, 1:4])
print(arr[:, 0])
```

```
Sortie :
[2 3 4 5]
[5 6 7]
[1 3 5 7]
[[2 3 4]
 [7 8 9]]
[1 6]
```

Travail demandé

12. Découpez `expeditions` en deux tableaux : `annee_1` (12 premiers mois) et `annee_2` (12 derniers mois). Extrayez aussi un mois sur trois dans `un_mois_sur_trois`.
13. Dans `stock`, extrayez : les trois premiers produits pour toutes les zones (`trois_premiers`) ; toute la colonne de la zone B (`zone_B`) ; le bloc des produits d'index 2 à 5 dans les zones B et C (`bloc`).
14. Dans `livraisons`, extrayez les délais (colonne d'index 3) des 10 premières livraisons dans `delais_debut`, puis les colonnes distance et délai (index 1 et 3, à l'aide d'un pas) des 5 premières livraisons dans `distance_delai`.

Visualisation attendue

Une fois les tâches terminées, exécutez la cellule de visualisation fournie dans le notebook. Vous devez obtenir :

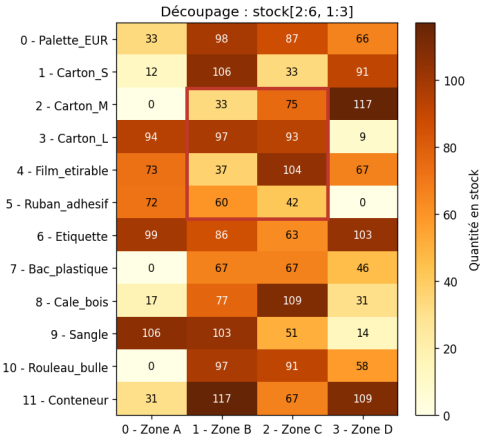


Figure : Carte du stock : la zone encadrée correspond à stock[2:6, 1:3]

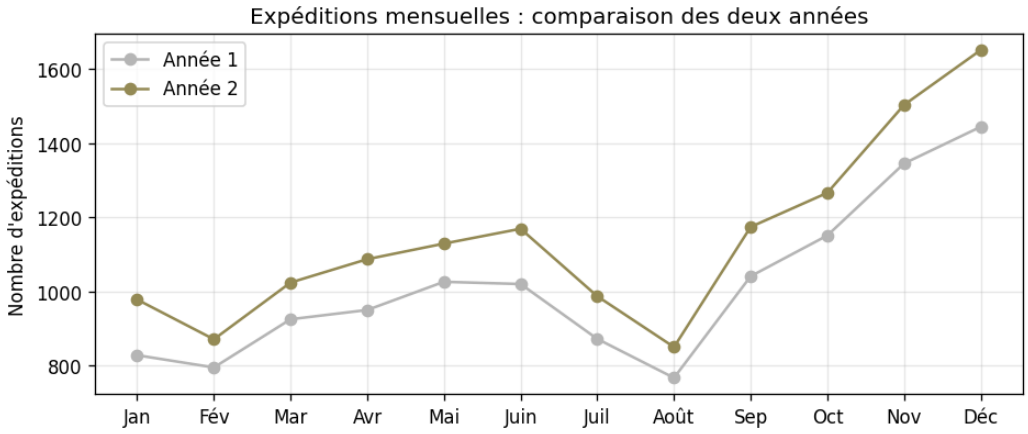


Figure : Les deux années extraites par découpage du tableau expeditions

Questions (à rédiger dans le notebook)

Q3.1 : Combien d'éléments contient `expeditions[2:10:2]` ? À quels index correspondent-ils ?

Q3.2 : Quelle est la différence entre `stock[1]` et `stock[:, 1]` ?

PARTIE 4 : Redimensionnement de tableaux (reshape) (25 min)

4.1 Rappel de cours

- La **structure** (`shape`) d'un tableau est le nombre d'éléments dans chaque dimension. L'attribut `shape` la renvoie sous forme de tuple : `(4, 3)` signifie 4 lignes et 3 colonnes.
- Redimensionner, c'est modifier cette structure avec `reshape()` : ajouter ou supprimer des dimensions, ou changer le nombre d'éléments par dimension.
- Condition : le **nombre total d'éléments doit rester le même**. 12 éléments peuvent devenir (4, 3), (3, 4), (2, 6) ou (2, 2, 3), mais pas (5, 2).

4.2 Exemple guidé

```
arr = np.array([1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12])
print(arr.shape)

newarr = arr.reshape(4, 3)
print(newarr)
print(newarr.shape)
print(newarr.ndim)
```

```
Sortie :
(12,)
[[ 1  2  3]
 [ 4  5  6]
 [ 7  8  9]
 [10 11 12]]
(4, 3)
2
```

Travail demandé

15. Affichez la structure (`shape`) des tableaux `expeditions`, `stock` et `livraisons`.
16. Transformez `expeditions` (24 mois) en un tableau 2-D `expeditions_par_annee` de 2 lignes (années) et 12 colonnes (mois). Affichez sa structure et son nombre de dimensions.
17. Transformez `expeditions` en un tableau 3-D `expeditions_trimestres` : 2 années, 4 trimestres par année, 3 mois par trimestre. Récupérez dans `valeur` le volume de l'année 2, trimestre 3, mois 2.
18. Transformez le tableau 2-D `stock` en un tableau 1-D `stock_plat` contenant toutes les quantités. Affichez sa structure.

Visualisation attendue

Une fois les tâches terminées, exécutez la cellule de visualisation fournie dans le notebook. Vous devez obtenir :

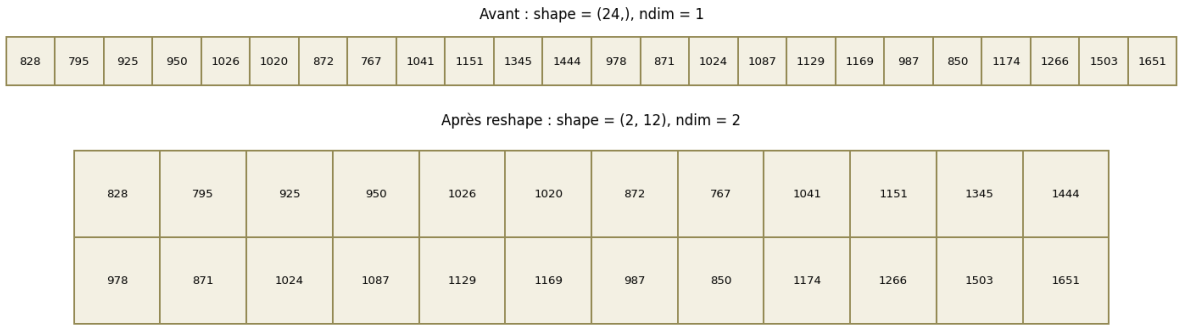


Figure : Le tableau expeditions avant et après reshape(2, 12)

Questions (à rédiger dans le notebook)

Q4.1 : Peut-on écrire `expeditions.reshape(5, 5)` ? Justifiez.

Q4.2 : Citez deux autres structures possibles pour un tableau de 24 éléments.

PARTIE 5 : Recherche dans les tableaux avec where() (30 min)

5.1 Rappel de cours

- On peut rechercher une valeur dans un tableau et obtenir les **index** où elle se trouve avec `np.where(condition)`.
- Le résultat est un **tuple** contenant un tableau d'index : `(array([3, 5, 6]),)`. Le tableau d'index s'obtient avec `resultat[0]`.
- La condition peut être une égalité (`arr == 4`) ou toute autre comparaison (`arr % 2 == 0`, `arr < 20`).
- Les index obtenus peuvent servir à récupérer les éléments correspondants : `arr[resultat]`.
- Forme complète : `np.where(condition, x, y)` renvoie un tableau de même taille contenant `x` là où la condition est vraie et `y` ailleurs.

5.2 Exemple guidé

```
# Recherche d'une valeur : où se trouve la valeur 4 ?
arr = np.array([1, 2, 3, 4, 5, 4, 4])
x = np.where(arr == 4)
print(x)

# Recherche par condition : où se trouvent les valeurs paires ?
arr = np.array([1, 2, 3, 4, 5, 6, 7, 8])
x = np.where(arr % 2 == 0)
print(x)
print(x[0])        # le tableau des index
print(arr[x])      # les valeurs situées à ces index

# Forme complète : 1 si la valeur dépasse 4, 0 sinon
etiquettes = np.where(arr > 4, 1, 0)
print(etiquettes)
```

```
Sortie :
(array([3, 5, 6]),)
(array([1, 3, 5, 7]),)
[1 3 5 7]
[2 4 6 8]
[0 0 0 0 1 1 1 1]
```

Travail demandé

19. Extrayez la colonne de la zone A dans `zone_A`. Recherchez dans `indices_rupture` les index des produits en rupture (quantité égale à 0), puis récupérez leurs noms dans `produits_rupture`.
20. Recherchez dans `indices_alerte` les index des produits de la zone A dont le stock est inférieur au seuil d'alerte de 20 unités. Calculez leur nombre dans `nombre_alertes` avec `len()`.
21. Extrayez la colonne des délais (index 3) de `livraisons` dans `delai`. Une livraison est en retard si son délai dépasse 48 h : recherchez les index des livraisons en retard dans `indices_retard` et comptez-les dans `nombre_retards`.
22. Avec la forme complète de `np.where`, construisez le tableau `statut_retard` qui vaut 1 pour une livraison en retard et 0 sinon. Affichez ses 20 premières valeurs.

Visualisation attendue

Une fois les tâches terminées, exécutez la cellule de visualisation fournie dans le notebook. Vous devez obtenir :

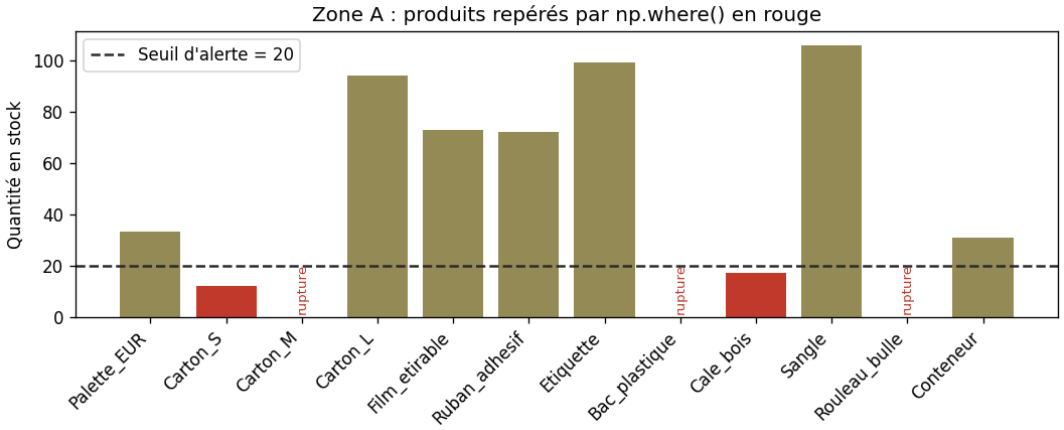


Figure : Stock de la zone A : produits sous le seuil d'alerte repérés par np.where()

Questions (à rédiger dans le notebook)

Q5.1 : Pourquoi faut-il écrire `indices_alerte[0]` pour compter les produits en alerte ?

Q5.2 : Quelle est la différence entre `np.where(delai > 48)` et `np.where(delai > 48, 1, 0)` ?

PARTIE 6 : Filtrage par tableau de booléens (30 min)

6.1 Rappel de cours

- **Filtrer**, c'est extraire certains éléments d'un tableau existant pour en créer un nouveau.
- Dans NumPy, on filtre un tableau à l'aide d'une liste (ou d'un tableau) de **booléens** de même longueur.
- Si la valeur à un index donné est `True`, l'élément est inclus dans le tableau filtré ; si elle est `False`, il est exclu.
- En pratique, le filtre est créé directement par une condition : `filtre = arr > 42` renvoie un tableau de booléens, que l'on applique avec `arr[filtre]`.

6.2 Exemple guidé

```
arr = np.array([41, 42, 43, 44])

# Filtre écrit à la main
x = [True, False, True, False]
newarr = arr[x]
print(newarr)

# Filtre créé par une condition
filtre = arr > 42
print(filtre)
print(arr[filtre])
```

```
Sortie :
[41 43]
[False False  True  True]
[43 44]
```

Travail demandé

23. Écrivez à la main le filtre `filtre_manuel` (liste de 4 booléens) qui conserve uniquement les zones A et C dans `capacites`. Appliquez-le pour obtenir `capacites_AC`.
24. Extrayez la colonne des distances (index 1) dans `distance`. Créez le filtre `filtre_retard` (délai supérieur à 48 h) puis obtenez `delais_en_retard` et `distances_en_retard`. Affichez le nombre de livraisons en retard.
25. Extrayez les colonnes poids (index 2) et coût (index 4) dans `poids` et `cout`. Filtrez les coûts des livraisons lourdes (poids supérieur à 800 kg) dans `couts_lourds`. Comparez le coût moyen des livraisons lourdes au coût moyen général (la méthode `.mean()` renvoie la moyenne d'un tableau).

Visualisation attendue

Une fois les tâches terminées, exécutez la cellule de visualisation fournie dans le notebook. Vous devez obtenir :

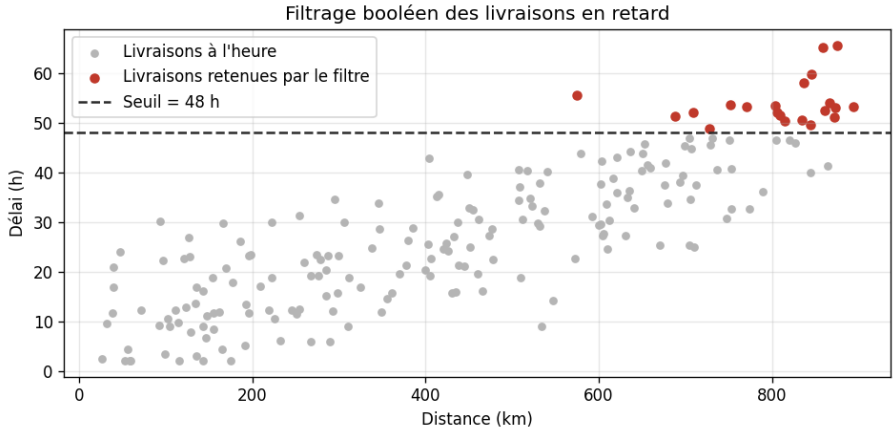


Figure : Livraisons en retard isolées par le filtre booléen

Questions (à rédiger dans le notebook)

Q6.1 : Quelle est la différence entre le résultat de `np.where(delai > 48)` et celui de `delai[delai > 48]` ?

Q6.2 : D'après le graphique, quelles livraisons sont le plus souvent en retard ?

PARTIE 7 : Synthèse : audit de l'entrepôt LogiStock (25 min)

Ces deux exercices, de difficulté moyenne, combinent toutes les notions du TP. Aucun exemple guidé n'est fourni : à vous de choisir les bons outils.

Travail demandé

26. **Audit de la zone D.** Extrayez la colonne de la zone D dans `zone_D`. Construisez le filtre `filtre_alerte_D` (stock inférieur à 20), puis obtenez les noms des produits concernés (`produits_alerte_D`) et leurs quantités (`quantites_alerte_D`). Recherchez enfin avec `np.where` l'index du ou des produits en rupture totale (`indices_rupture_D`).
27. **Analyse des tournées.** Les 200 livraisons sont réparties en 20 tournées de 10 livraisons. Redimensionnez `delai` en `delai_tournees` (20 lignes, 10 colonnes). Extrayez la tournée d'index 4 (`tournee_4`), puis ses délais en retard (`retards_tournee_4`). Extrayez les 5 premières livraisons des 3 premières tournées (`debut_tournees`). Construisez enfin `statut_tournees`, qui vaut 1 pour un retard et 0 sinon.

Visualisation attendue

Une fois les tâches terminées, exécutez la cellule de visualisation fournie dans le notebook. Vous devez obtenir :

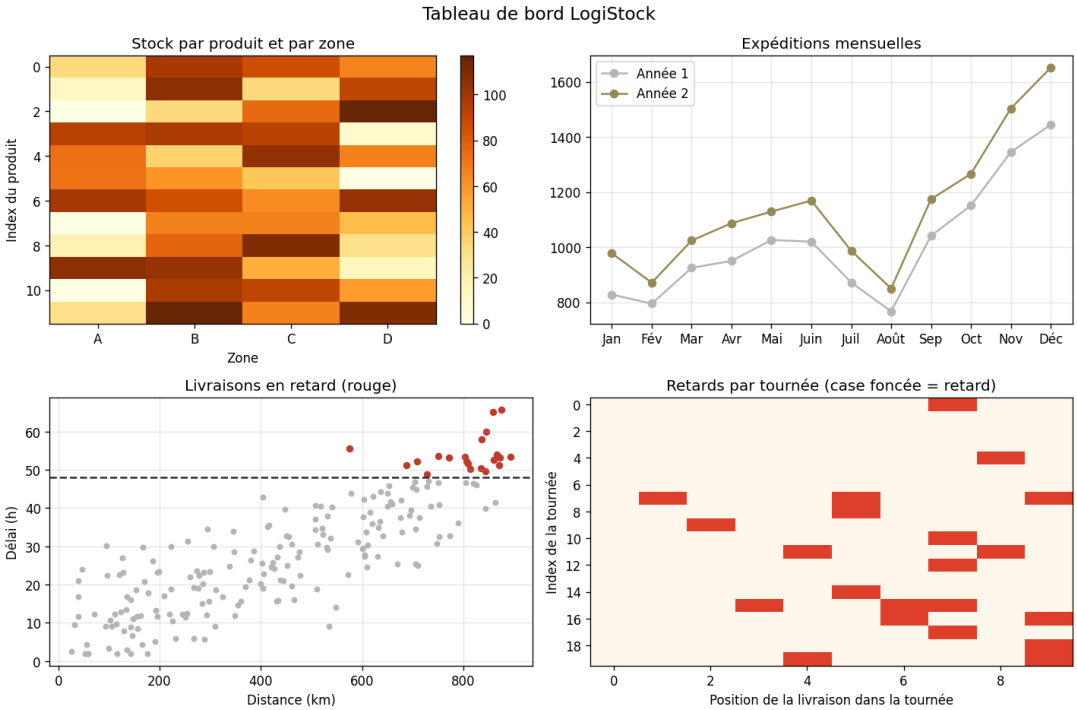


Figure : Tableau de bord LogiStock construit à partir de vos tableaux

Questions (à rédiger dans le notebook)

Q7.1 : En observant le tableau de bord, formulez trois constats utiles au responsable de l'entrepôt.

LIVRABLES ATTENDUS

- **Notebook complété** (`.ipynb`) : toutes les cellules `# A COMPLETER` renseignées et exécutées sans erreur, visualisations affichées.
- **Réponses aux questions** Q0 à Q7, rédigées dans les cellules de texte prévues à cet effet.

Annexe : aide-mémoire NumPy

Instruction	Rôle
`np.array(liste)`	Crée un tableau ndarray à partir d'une liste
`arr.ndim`	Nombre de dimensions du tableau
`arr.shape`	Structure : nombre d'éléments dans chaque dimension
`arr[i] arr[i, j] arr[i, j, k]`	Indexation en 1-D, 2-D et 3-D
`arr[début:fin:pas]`	Découpage (index de fin exclu)
`arr[:, j]`	Toute la colonne d'index j d'un tableau 2-D
`arr.reshape(a, b)`	Redimensionnement (même nombre total d'éléments)
`np.where(condition)`	Index des éléments qui vérifient la condition (tuple)
`np.where(condition, x, y)`	x là où la condition est vraie, y ailleurs
`arr[filtre]`	Filtrage par un tableau de booléens